BANK OF AGRICULTURE

Customer Credit Check API

Integration Guide

For partner teams connecting their systems to the Bank of Agriculture's customer credit check: how to get access, submit a check, receive the result and go live.

Version	1.0
Date	6 November 2026
Audience	Partner integration teams

Contents

1. Overview

2. Getting your credentials

3. Quick start

4. Authentication

5. Endpoints at a glance

6. Submitting a check

7. Receiving the result by callback

8. Fetching status, results and history

9. Understanding the result

10. Errors and limits

11. Charges

12. Security and data handling

13. Going live

14. Getting help

Appendix A. A complete result

Appendix B. Values you will meet

1. Overview

The Customer Credit Check API tells your system what is known about a person, from their BVN, in one request. It is read-only. It never changes an account, a loan or a case at the bank.

A check can return up to seven sections. You choose which ones you need.

Section	Name in the API	What it tells you
Identity	`identity`	Who the BVN belongs to, and whether the person is on a watch list.
Credit bureau	`creditBureau`	The person's loans with other lenders, their repayment record and their credit score.
Fraud check	`fraudCheck`	A fraud risk score and band.
Bank accounts	`bankAccounts`	The accounts the person holds at the Bank of Agriculture, with balances and activity.
Bank loans	`bankLoans`	The loans the person has with the Bank of Agriculture, and how they are performing.
CRMS facilities	`crmsFacilities`	Every credit case at the bank the person is tied to, as borrower, guarantor or otherwise.
AI advisory	`aiAdvisory`	A score, a risk rating and a recommendation drawn from all of the above, with the reasons.

How a check works

A check asks several outside providers for data, so it can take from a few seconds to a couple of minutes. For that reason the API does not make you wait.

1. **You submit a check.** The API answers at once with a job id.
2. **The bank runs the check** in the background.
3. **The bank posts the result to your callback URL**, signed so you can prove it came from the bank.
4. **You can also fetch the result yourself** at any time, using the job id.

You do not have to use callbacks. If you prefer, submit the check and then ask for the result until it is ready.

What you need

You need	Where it comes from
The base URL of the API, for test and for production	Your contact at the bank
An API key	The setup link the bank emails you (section 2)
A callback signing secret	The same setup link
A callback URL on your side, if you want callbacks	You build it and tell the bank its address (section 7)

Throughout this guide, `{base-url}` stands for the base URL the bank gives you.

2. Getting your credentials

The bank never sends your API key or signing secret by email or by phone. You collect them yourself from a one-time link.

Before the link is sent

Give your contact at the bank:

- **One contact email address.** Use a secure mailbox that your integration team controls. The setup link goes to this address and nowhere else.
- **Your callback URL**, if you want results posted to you. It must use `https` in production.
- **Any other hosts** your callbacks may need to go to, for example a staging server.
- **The sections you need** from the table in section 1.

Collecting them

1. The bank sends an email to your contact address with a link. The email contains no key and no secret.
2. Open the link. The page tells you what it is about to issue. Nothing has changed yet.
3. Have a safe place ready, such as your secrets manager. Then press **Show my credentials**.
4. The page shows your API key, your callback signing secret, or both. Copy them and store them.

Rules to know

- **The link works once.** After you press the button, the page cannot be opened again.
- **The link expires.** The email says when. It is normally 24 hours.
- **The bank cannot show you the API key later.** The bank stores only a fingerprint of it. If you lose it, ask for a new link.

- **Your current credentials keep working until you press the button.** From that moment only the new ones are accepted. Plan the switch so your systems are ready.
- **A newer link cancels an older one** that has not been opened.
- **If the page says the link has already been used and it was not you, tell the bank at once.** The bank will replace your credentials.
- **If you were not expecting the email, do not open the link.** Tell your contact at the bank.

The same link is how credentials are replaced later, for example after a staff change on your side or a suspected leak. Ask the bank to send a new one.

3. Quick start

This is the shortest path to a first result. It uses `curl`. Replace `{base-url}` and the key with your own.

Step 1. Submit a check. Only `requestId` and `bvn` are required.

```
curl -X POST "{base-url}/api/v1/customer-credit-check" \
  -H "Content-Type: application/json" \
  -H "X-Api-Key: YOUR_API_KEY" \
  -d '{"requestId":"c7f1e2a0-9b1d-4e4e-8d0a-7b9f3c2d1a55","bvn":"22212344321"}'
```

Step 2. Read the answer. You get `202 Accepted` and a job id straight away.

```
{
  "jobId": "7a2f1c9e4b8d4f0a9c3e5d1b2a6f8e7c",
  "requestId": "c7f1e2a0-9b1d-4e4e-8d0a-7b9f3c2d1a55",
  "status": "QUEUED",
  "statusUrl": "/api/v1/customer-credit-check/7a2f1c9e4b8d4f0a9c3e5d1b2a6f8e7c",
  "queuedAt": "2026-11-05T09:42:17Z",
  "callbackRegistered": true
}
```

Step 3. Get the result. Either wait for the callback (section 7), or ask for it:

```
curl "{base-url}/api/v1/customer-credit-check/7a2f1c9e4b8d4f0a9c3e5d1b2a6f8e7c" \
  -H "X-Api-Key: YOUR_API_KEY"
```

When `status` is `COMPLETED`, the reply carries the full `result`. Until then, ask again after a few seconds.

4. Authentication

Send your API key in the `X-Api-Key` header on every call.

```
X-Api-Key: YOUR_API_KEY
```

- The key identifies your organisation. Every check is recorded against it.
- You can only see your own checks.
- A missing, wrong or switched-off key gets `401`.
- Always call the API over `https` in production.
- Keep the key on your server. Never put it in a mobile app, a web page or source control.

5. Endpoints at a glance

What you want	Method and path	Answer
Submit a check	`POST /api/v1/customer-credit-check`	`202 Accepted` with a job id
Status and result of one check	`GET /api/v1/customer-credit-check/{jobId}`	`200` with the status, and the result when finished
List your past checks	`POST /api/v1/customer-credit-check/history`	`200` with a page of checks

All request and response bodies are JSON. Send `Content-Type: application/json` with every body. All times are UTC in ISO 8601 format, for example `2026-11-05T09:42:17Z`.

The bank also publishes an interactive reference for the API, where you can read every field and try requests. Where it is enabled, it is at `{base-url}/scalar/v1`.

6. Submitting a check

`POST /api/v1/customer-credit-check`

The smallest request

```
{
  "requestId": "c7f1e2a0-9b1d-4e4e-8d0a-7b9f3c2d1a55",
  "bvn": "22212344321"
}
```

This runs every section you are entitled to.

A request with every optional field

```
{
  "requestId": "c7f1e2a0-9b1d-4e4e-8d0a-7b9f3c2d1a55",
  "bvn": "22212344321",
  "nubans": ["0123455678"],
  "sections": ["identity", "creditBureau", "fraudCheck", "bankAccounts", "bankLoans",
"crmsFacilities", "aiAdvisory"],
  "callbackUrl": "https://partner.example.com/crms/customer-credit-check",
  "options": {
    "activityWindowMonths": 12,
    "includeScore": true,
    "runAmlCheck": false,
    "loanAmount": 500000,
    "tenorMonths": 12
  }
}
```

Fields

Field	Required	Rules	What it does
`requestId`	Yes	Up to 100 characters. Unique for each check you make.	Your own reference for the check. It is returned with the result. It also protects you from running the same check twice (see below).
`bvn`	Yes	11 digits. Spaces, hyphens and dots are ignored.	The person to check.

Field	Required	Rules	What it does
`nubans`	No	Up to 5 account numbers, 10 digits each.	Accounts are found from the BVN, so you do not need this. Naming accounts puts them first in line when account activity is read.
`sections`	No	Any of the seven names in section 1.	The sections to run. Leave it out to run all of them.
`callbackUrl`	No	An absolute URL of up to 500 characters, on a host the bank has allowed for you. `https` in production. No user name or password in the URL.	Where to post this result. Leave it out to use the callback URL the bank holds for you.
`options.activityWindowMonths`	No	1 to 24. Default 12.	How many months of account activity to read.
`options.includeScore`	No	Default `true`.	Also fetch the bureau's credit score.
`options.runAmlCheck`	No	Default `false`.	Add an anti-money-laundering screen to the fraud check.
`options.loanAmount`	No	Greater than zero.	The loan the person is asking you for. The advisory and the fraud check use it.
`options.tenorMonths`	No	1 to 120.	The term of that loan in months.

Tip. If the person is applying for a specific loan, send `loanAmount` and `tenorMonths`. The advisory then judges whether that loan is affordable and can suggest an amount. Without them, a nominal loan is assumed and the advisory says so.

The answer

`202 Accepted`. The `Location` header carries the status URL as well.

Field	Meaning
`jobId`	The bank's id for this check. Use it to fetch the result.
`requestId`	Your own reference, returned to you.
`status`	`QUEUED` for a new check.
`statusUrl`	The path to fetch the status and result.
`queuedAt`	When the check was accepted.
`callbackRegistered`	`true` if a result will be posted to a callback URL. `false` means you must fetch it.

Sending the same check twice

`requestId` is your safety net. If you send the same `requestId` again within 24 hours, the API returns the existing job instead of running a second check. You get another `202` with the job's current status.

This means it is always safe to retry a submission that timed out or failed on the network. Use the same `requestId` when you retry.

A new `requestId` for the same BVN is a new check.

Sections you are not entitled to

Asking for a section the bank has not enabled for you is not an error. The check runs, and that section comes back with `status: NOT_ALLOWED`.

7. Receiving the result by callback

When a check finishes, the bank sends one `POST` to your callback URL.

What you receive

```
{
  "eventType": "CUSTOMER_CREDIT_CHECK_COMPLETED",
  "eventId": "evt_3f9c1a7e2b5d4c8e9a1f6b2d7c3e8a4f",
  "jobId": "7a2f1c9e4b8d4f0a9c3e5d1b2a6f8e7c",
  "requestId": "c7f1e2a0-9b1d-4e4e-8d0a-7b9f3c2d1a55",
  "status": "COMPLETED",
  "occurredAt": "2026-11-05T09:43:01Z",
  "statusUrl": "/api/v1/customer-credit-check/7a2f1c9e4b8d4f0a9c3e5d1b2a6f8e7c",
  "error": null,
  "result": { }
}
```

Field	Meaning
`eventType`	`CUSTOMER_CREDIT_CHECK_COMPLETED` or `CUSTOMER_CREDIT_CHECK_FAILED`.
`eventId`	The id of this event. It stays the same on every retry.
`jobId`, `requestId`	The check this is about.
`status`	`COMPLETED` or `FAILED`.
`result`	The full result, described in section 9. Present only when completed.
`error`	`{ "code", "message" }`. Present only when failed.

The request carries these headers:

Header	Value
`Content-Type`	`application/json`
`X-CRMS-Event-Id`	The same as `eventId`.
`X-CRMS-Event-Type`	The same as `eventType`.
`X-CRMS-Timestamp`	When the request was signed, in Unix seconds.
`X-CRMS-Signature-V2`	`sha256=` followed by the signature in lowercase hex.

What your endpoint must do

1. **Verify the signature** before you trust anything in the body (see below).

2. **Answer with any `2xx` status within 15 seconds.** Store the result and return. Do your own processing afterwards.
3. **Ignore repeats.** If you have already handled an `eventId`, answer `2xx` and do nothing more.

Retries

If your endpoint answers anything other than `2xx`, or does not answer within 15 seconds, the bank tries again after 1, 3, 7, 15 and 15 minutes. That is six attempts in all.

After the last attempt the bank stops sending. The result is not lost. You can fetch it at any time from the status URL.

Verifying the signature

The signature proves the request came from the bank and was not altered. It is an HMAC-SHA256, using your callback signing secret as the key, over this exact text:

```
{X-CRMS-Timestamp}.{raw request body}
```

That is the timestamp, a full stop, and the body exactly as it arrived.

1. Read `X-CRMS-Timestamp`. Reject the request if it is more than five minutes away from your clock.
2. Compute HMAC-SHA256 over the timestamp, a full stop and the raw body. Use your signing secret as the key. Encode the result as lowercase hex.
3. Compare it with the part of `X-CRMS-Signature-V2` after `sha256=`. Use a constant-time comparison.
4. If they differ, answer `401` and stop.

Use the raw body. Compute the signature over the bytes you received, before any JSON parsing. If your framework parses the body and you turn it back into text, the signature will not match.

Node.js

```
const crypto = require('crypto');

function isFromTheBank(timestamp, rawBody, signatureHeader, secret) {
  const age = Math.abs(Date.now() / 1000 - Number(timestamp));
  if (!Number.isFinite(age) || age > 300) return false;

  const expected = crypto.createHmac('sha256', secret)
    .update(`${timestamp}.${rawBody}`)
    .digest('hex');
  const given = String(signatureHeader || '').replace(/^sha256=/, '').toLowerCase();

  const a = Buffer.from(expected), b = Buffer.from(given);
```

```
  return a.length === b.length && crypto.timingSafeEqual(a, b);
}

// Express: keep the raw body for this route.
app.post('/crms/customer-credit-check',
  express.raw({ type: 'application/json' }),
  (req, res) => {
    const raw = req.body.toString('utf8');
    const ok = isFromTheBank(req.get('X-CRMS-Timestamp'), raw,
      req.get('X-CRMS-Signature-V2'), process.env.CRMS_SIGNING_SECRET);
    if (!ok) return res.sendStatus(401);

    const event = JSON.parse(raw);
    saveForProcessing(event);   // store it, then answer quickly
    res.sendStatus(200);
  });
```

Python

```
import hashlib, hmac, time

def is_from_the_bank(timestamp: str, raw_body: bytes, signature_header: str, secret: str) -> bool:
    try:
        if abs(time.time() - int(timestamp)) > 300:
            return False
    except (TypeError, ValueError):
        return False

    message = timestamp.encode() + b"." + raw_body
    expected = hmac.new(secret.encode(), message, hashlib.sha256).hexdigest()
    given = (signature_header or "").removeprefix("sha256=").lower()
    return hmac.compare_digest(expected, given)

# Flask
@app.post("/crms/customer-credit-check")
def crms_callback():
    raw = request.get_data()   # the raw bytes, before parsing
    if not is_from_the_bank(request.headers.get("X-CRMS-Timestamp"), raw,
                            request.headers.get("X-CRMS-Signature-V2"), SIGNING_SECRET):
        return "", 401
    save_for_processing(json.loads(raw))
    return "", 200
```

C#

```
static bool IsFromTheBank(string timestamp, string rawBody, string signatureHeader, string secret)
{
    if (!long.TryParse(timestamp, out var sentAt)) return false;
    if (Math.Abs(DateTimeOffset.UtcNow.ToUnixTimeSeconds() - sentAt) > 300) return false;

    using var hmac = new HMACSHA256(Encoding.UTF8.GetBytes(secret));
    var hash = hmac.ComputeHash(Encoding.UTF8.GetBytes($"{timestamp}.{rawBody}"));
    var expected = Convert.ToHexString(hash).ToLowerInvariant();

    var given = (signatureHeader ?? "").Trim();
    if (given.StartsWith("sha256=")) given = given[7..];

    return CryptographicOperations.FixedTimeEquals(
        Encoding.UTF8.GetBytes(expected),
        Encoding.UTF8.GetBytes(given.ToLowerInvariant()));
}
```

PHP

```
function isFromTheBank(string $timestamp, string $rawBody, string $signatureHeader, string $secret):
bool
{
    if (!ctype_digit($timestamp) || abs(time() - (int) $timestamp) > 300) {
        return false;
    }
    $expected = hash_hmac('sha256', $timestamp . '.' . $rawBody, $secret);
    $given = strtolower(preg_replace('/^sha256=/', '', trim($signatureHeader)));
    return hash_equals($expected, $given);
}

$raw = file_get_contents('php://input');   // the raw body
$ok = isFromTheBank($_SERVER['HTTP_X_CRMS_TIMESTAMP'] ?? '', $raw,
                    $_SERVER['HTTP_X_CRMS_SIGNATURE_V2'] ?? '', getenv('CRMS_SIGNING_SECRET'));
http_response_code($ok ? 200 : 401);
```

Check your code with this example

Use these values to test your function. They are examples only.

Input	Value
Signing secret	`crms_whsec_EXAMPLE_ONLY_do_not_use_in_production`
`X-CRMS-Timestamp`	`1793871781`
Raw body	`{"eventType":"CUSTOMER_CREDIT_CHECK_COMPLETED","eventId":"evt_example"}`

The text that is signed is:

```
1793871781.{"eventType":"CUSTOMER_CREDIT_CHECK_COMPLETED","eventId":"evt_example"}
```

The header you should arrive at is:

```
X-CRMS-Signature-V2: sha256=4641df720103a795d4a2d97bee48371ebd17ba45cf763fc2516731b6b00bb188
```

When you run this test, switch off the five-minute check, because the example timestamp is in the past.

Choosing a callback URL per request

You can send a different `callbackUrl` with a request, for example to route results to another service of yours. The host must be one the bank has allowed for you. Any other host is refused with `400`. This protects you: a leaked key cannot be used to send a person's credit data somewhere else.

8. Fetching status, results and history

One check

`GET /api/v1/customer-credit-check/{jobId}`

```
{
  "jobId": "7a2f1c9e4b8d4f0a9c3e5d1b2a6f8e7c",
  "requestId": "c7f1e2a0-9b1d-4e4e-8d0a-7b9f3c2d1a55",
  "status": "COMPLETED",
  "queuedAt": "2026-11-05T09:42:17Z",
  "startedAt": "2026-11-05T09:42:18Z",
  "completedAt": "2026-11-05T09:43:01Z",
  "sections": ["identity", "creditBureau", "fraudCheck", "bankAccounts", "bankLoans",
"crmsFacilities", "aiAdvisory"],
  "error": null,
  "callback": {
    "status": "DELIVERED",
    "attempts": 1,
    "deliveredAt": "2026-11-05T09:43:02Z",
    "lastStatusCode": 200,
    "nextAttemptAt": null
  },
  "result": { }
}
```


status	Meaning	What to do
`QUEUED`	Waiting to run.	Ask again shortly.
`PROCESSING`	Running now.	Ask again shortly.
`COMPLETED`	Finished. `result` is present.	Use the result.
`FAILED`	Could not be run. `error` says why.	Submit again with a new `requestId`.

callback.status	Meaning
`NOT_REQUESTED`	No callback URL applies to this check.
`PENDING`	Not yet delivered, or waiting for the next attempt.
`DELIVERED`	Your endpoint answered `2xx`.
`FAILED`	Every attempt was refused. Fetch the result from this endpoint instead.

If you poll. Wait about 5 seconds between requests. Most checks finish within a minute or two. Stop when the status is `COMPLETED` or `FAILED`.

Results are kept, so you can fetch the same result again later.

Your past checks

`POST /api/v1/customer-credit-check/history`

This is a `POST` so that a BVN never appears in a URL.

```
{
  "from": "2026-11-01",
  "to": "2026-11-30",
  "status": "COMPLETED",
  "requestId": null,
  "bvn": null,
  "page": 1,
  "pageSize": 50
}
```

Every field is optional. An empty body `{}` returns your most recent checks.

Field	Meaning
`from`, `to`	Limit the list to checks submitted between these dates.
`status`	`QUEUED`, `PROCESSING`, `COMPLETED` or `FAILED`.
`requestId`	Find the check you submitted under this reference.
`bvn`	Find your checks on this person.
`page`	Starts at 1.
`pageSize`	1 to 100.

The reply lists checks, newest first, without their results:

```
{
  "page": 1,
  "pageSize": 50,
  "total": 132,
  "items": [
    {
      "jobId": "7a2f1c9e4b8d4f0a9c3e5d1b2a6f8e7c",
      "requestId": "c7f1e2a0-9b1d-4e4e-8d0a-7b9f3c2d1a55",
      "status": "COMPLETED",
      "queuedAt": "2026-11-05T09:42:17Z",
      "completedAt": "2026-11-05T09:43:01Z",
      "sections": ["identity", "creditBureau"],
      "bvnLast4": "4321",
      "callback": { "status": "DELIVERED" }
    }
  ]
}
```

To get the result of any item, fetch it by its `jobId`.

9. Understanding the result

The result is the same whether it arrives in a callback or you fetch it. A complete example is in Appendix A.

The outline

Field	Meaning
`requestId`	Your reference.
`dataAsOf`	When the data was gathered.
`consumer`	Your organisation's name as the bank holds it.
`subject.bvnLast4`	The last four digits of the BVN checked.
`subject.nubansChecked`	The account numbers you named, masked.
`sectionsRun`	The sections that actually ran.

Field	Meaning
`identity`, `creditBureau`, `fraudCheck`, `bankAccounts`, `bankLoans`, `crmsFacilities`, `aiAdvisory`	The seven sections. Each is always present.
`coverage`	A plain sentence per section on what the data does and does not cover.
`warnings`	Points worth a person's attention, in plain words.

Check each section's status first

Every section carries its own `status`. One section failing never blanks another. Always read the status before the data.

status	Meaning	How to treat it
`OK`	The section answered.	Use the data.
`NO_RECORD`	The source answered and holds nothing on this person.	A real answer. For example, the person has no credit history at the bureau.
`UNAVAILABLE`	The source failed or did not answer in time. `reason` says which.	Not an answer about the person. Run the check again later if you need this section.
`NOT_REQUESTED`	You did not ask for it.	Nothing to do.
`NOT_ALLOWED`	The bank has not enabled it for you.	Speak to the bank if you need it.

Do not treat `UNAVAILABLE` as `NO_RECORD`. "We could not find out" is different from "there is nothing to find".

Identity

Field	Meaning
`fullName`, `dateOfBirth`, `gender`	As held against the BVN.
`phoneNumber`	Partly masked.
`stateOfResidence`, `enrollmentBank`, `registrationDate`	BVN enrolment details.
`watchListed`	`true` if the BVN is on a watch list.
`source`	Where the identity data came from.

Credit bureau

Field	Meaning
`bureau`	The bureau that answered: `FIRST_CENTRAL`, `CRC` or `CREDIT_REGISTRY`.
`bureausSkipped`	Bureaus that were tried and failed before this one answered.
`reused`	`true` if a recent stored report was used instead of a new one. `reportDate` is then the date of that report.
`score`	`value`, `band` and `source` of the credit score. `null` if no score was requested or available.
`summary`	Totals: number of loans, lenders, active, closed, performing and delinquent loans, the amounts borrowed, outstanding and overdue, and the worst days overdue.
`loans`	One entry per loan: lender, type, amounts, status, dates, tenor.
`recentEnquiries`	Other lenders that recently looked the person up.

Bureaus are tried in order. If one fails, the next is tried. If one answers that it holds no record, that is taken as the answer and the section is `NO_RECORD`.

Fraud check

Field	Meaning
`riskScore`	0 to 100. Higher is riskier.
`riskBand`	`LOW` (below 30), `MEDIUM` (30 to 69) or `HIGH` (70 and above).
`recommendation`	The fraud provider's advice in words.
`amlChecked`	`true` if an anti-money-laundering screen was included.
`financialAnalysis.assessed`	`true` only when you sent both `loanAmount` and `tenorMonths`.

Bank accounts

Field	Meaning
`bvnMatch`	`MATCH` when the bank holds a customer with this BVN.
`clientStatus`, `kycLevel`, `customerSince`	The customer's standing with the bank.
`accounts[].accountNumberLast4`	The last four digits of the account number.
`accounts[].productName`, `accountType`	The kind of account, for example `SAVINGS` or `FIXED DEPOSIT`.
`accounts[].status`	`ACTIVE`, `DORMANT`, `CLOSED`, `INACTIVE` or `UNKNOWN`. `rawStatus` is the bank's own word.
`accounts[].balance`, `currency`	The current balance.
`accounts[].openedDate`, `lastTransaction`	Account dates.
`accounts[].activity`	Over the window you asked for: number of transactions, total credits and total debits. `null` if activity was not read for this account.

Activity is read for a limited number of accounts per check. The accounts you name in `nubans` come first, then active accounts, then the largest balances. `reason` says how many were read.

Bank loans

Field	Meaning
`loans[].loanAccountLast4`	The last four digits of the loan account.
`loans[].productName`	The loan product.
`loans[].principal`, `outstanding`, `overdueAmount`	Amounts in naira.
`loans[].daysPastDue`	The age in days of the oldest unpaid instalment.
`loans[].status`	`RUNNING`, `OVERDUE`, `PAID`, `WRITTEN_OFF`, `IN_PROCESS` or `UNKNOWN`. `rawStatus` is the bank's own word.
`loans[].disbursedDate`, `maturityDate`	Loan dates.
`loans[].nextDueDate`, `nextDueAmount`	The next instalment.

The same loan status words are used for bureau loans, so you can treat both lists alike.

CRMS facilities

This lists every credit case at the bank that the person is connected to.

Field	Meaning
`matchStatus`	`MATCH` when the person appears on at least one case.
`summary.direct`	Cases where the person is the borrower.
`summary.indirect`	Cases where the person is connected in another role.
`summary.activeOutstanding`	The total still owed on active cases.
`relationships[].role`	`BORROWER`, `GUARANTOR`, `DIRECTOR_OR_OWNER`, `ACCOUNT_SIGNATORY`, `NEXT_OF_KIN_OR_REFEREE` or `OTHER`.
`relationships[].caseReference`, `facilityType`, `obligor`	Which case, what kind, and who owes it.
`relationships[].status`, `amountApproved`, `outstanding`, `daysPastDue`	How the facility stands.

AI advisory

The advisory runs last. It scores what the other sections found and explains its reasons. Fixed rules decide every number. The written narrative only explains them.

Field	Meaning
`overallScore`	0 to 100. Higher is better.
`overallRating`	The risk level: `VeryLow`, `Low`, `Medium`, `High` or `VeryHigh`.
`recommendation`	`StrongApprove`, `Approve`, `ApproveWithConditions`, `Refer` or `Decline`.
`categories`	The four scores that make up the overall score (below).
`redFlags`	Serious concerns, in plain words.
`conditions`	Conditions suggested before lending.
`narrative`	A written summary: `summary`, `strengths`, `weaknesses`, `mitigatingFactors`, `keyRisks`.
`proposal`	The loan the advisory measured against. `supplied` is `true` if you sent it. `recommendedAmount` is present only when you did.
`basis`	What the score was drawn from.

Field	Meaning
`narrativeSource`	`LLM` if a language model wrote the narrative, `TEMPLATE` if it was assembled from the rules' own reasons.

The four categories:

Category	What it looks at	Usual weight
`CreditHistory`	The bureau report and the person's loans with the bank.	40%
`RepaymentCapacity`	The proposed instalment and existing instalments, against average monthly inflow.	25%
`CashflowStability`	Account activity over the window.	20%
`IdentityAndFraud`	BVN verification, watch list, fraud score and relationship with the bank.	15%

Each category has `assessed`, `score`, `weight`, `rating`, `rationale`, `positives` and `concerns`. A category the data cannot support has `assessed: false` and no score. The overall score is then worked out from the assessed categories only.

The advisory is an aid, not a decision. It summarises the evidence for the person who decides. Your own credit policy applies.

No name, BVN, account number, phone number or date of birth is sent to the language model. It receives figures and flags only.

What the API cannot tell you

- **Income.** The API does not know the person's income.
- **Regulatory classification** of a loan.
- **The bureau's full PDF report.** You receive the structured data instead.

10. Errors and limits

When a request is refused

Errors come back as JSON with an `error` code and a `message`. A validation error carries a list of `messages` instead, one for each problem.

```
{ "error": "validation_failed", "messages": ["bvn must be 11 digits."] }
```

HTTP	error	Meaning	What to do
400	`invalid_payload`	The body is not valid JSON.	Fix the request.
400	`validation_failed`	A field is wrong. `messages` lists each problem.	Fix the request. The message names the field, never its value.
401	`unauthorized`	The key is missing, wrong or switched off.	Check the key. If it was replaced, use the new one.
404	`not_found`	No check with that id belongs to you.	Check the job id.
429	`rate_limited`	You sent too many requests in a minute.	Wait for the number of seconds in the `Retry-After` header, then retry.
429	`queue_full`	You have too many checks waiting or running.	Wait for some to finish, then retry.
503	`unavailable`	The check could not be queued, or the stored result could not be read.	Retry shortly. Nothing was queued.

When a check fails after it was accepted

The job's `status` is `FAILED` and `error` says why. The same appears in a `CUSTOMER_CREDIT_CHECK_FAILED` callback.

error.code	Meaning
`check_failed`	The check could not be completed.
`timed_out`	The check took too long and was stopped.
`interrupted`	The check was interrupted more than once by a restart.
`consumer_inactive`	Your access was switched off before the check ran.

Submit the check again with a new `requestId`. If it keeps failing, contact the bank.

Limits

Limit	What happens
Requests per minute	Set by the bank for your organisation. Beyond it you get `429 rate_limited`.
Checks waiting or running at once	Set by the bank. Beyond it you get `429 queue_full`.
Account numbers per request	5
Page size in history	100
Callback response time	15 seconds

Retrying safely

- **A submission that failed on the network or timed out:** send it again with the same `requestId`. You will not run the check twice.
- A `429`: wait, then retry. Honour `Retry-After` when it is present.
- A `503`: retry after a short pause, with the same `requestId`.
- A `400` or `401`: do not retry unchanged. Fix the request or the key first.

11. Charges

Your agreement with the bank sets out what you pay. These points help you control it.

- **Ask only for the sections you need.** Bureau, fraud, identity and advisory lookups each cost money.
- **The credit score is a separate lookup.** Set `options.includeScore` to `false` if you do not need it.
- **A repeated `requestId` within 24 hours is not a new check.**
- **A failed check is not charged.**
- **Fetching a stored result is free.** You can read a result again as often as you like.
- **Bureau reports can be reused.** The bank can let a recent bureau report on the same person be reused instead of buying a new one. `creditBureau.reused` tells you when that happened.

12. Security and data handling

- **Store the API key and signing secret in a secrets manager.** Not in source code, not in email, not in a shared document.
- **Use `https`** for every call and for your callback URL.

- **Verify every callback** before using it. Reject any that fail.
- **Never put a BVN in a URL or a log.** The API is built so that you never need to.
- **Sensitive values are masked.** Account numbers come back as their last four digits. Phone numbers are partly masked. The full BVN is never returned.
- **Limit who can see results.** They contain personal financial data.
- **Run a check only for a person and a purpose that your agreement with the bank covers.**
- **Tell the bank at once** if a key or secret may have leaked. Ask for a new setup link. The old credentials stop working the moment the new ones are collected.

13. Going live

Work through this list in the test environment first, then again in production. Test and production have separate base URLs and separate credentials.

1. You can collect credentials from a setup link and store them safely.
2. A check with only `requestId` and `bvn` returns `202`.
3. Sending the same `requestId` again returns the same `jobId`.
4. Your callback endpoint verifies the signature and rejects a request with a wrong one.
5. Your callback endpoint answers `2xx` within 15 seconds.
6. A repeated `eventId` is handled once.
7. You can fetch a result by `jobId` when a callback is missed.
8. Your code reads each section's `status` before its data, and treats `UNAVAILABLE` differently from `NO_RECORD`.
9. Your code backs off on `429` and retries a `503`.
10. A missing or wrong key gives `401`, and your system reports it clearly.
11. Keys and secrets are not in source control or logs.

14. Getting help

Speak to your contact at the bank. To help them find the check quickly, send:

- your `requestId` and the `jobId`
- the date and time of the request
- the HTTP status and the `error` code you received

Never send an API key, a signing secret or a full BVN in an email or a support message.

Appendix A. A complete result


```
{
  "requestId": "c7f1e2a0-9b1d-4e4e-8d0a-7b9f3c2d1a55",
  "dataAsOf": "2026-11-04T09:42:17Z",
  "consumer": "AgriPay Ltd",
  "subject": { "bvnLast4": "4321", "nubansChecked": ["****5678"] },
  "sectionsRun": ["identity", "creditBureau", "fraudCheck", "bankAccounts", "bankLoans", "crmsFacilities",
"aiAdvisory"],

  "identity": {
    "status": "OK",
    "source": "SmartComply BVN Advanced",
    "fullName": "ADEBAYO MUSA OKORO",
    "dateOfBirth": "1985-03-14",
    "gender": "Male",
    "phoneNumber": "0803****123",
    "stateOfResidence": "Kaduna",
    "enrollmentBank": "Bank of Agriculture",
    "registrationDate": "2014-08-22",
    "watchListed": false
  },

  "creditBureau": {
    "status": "OK",
    "bureau": "FIRST_CENTRAL",
    "bureausSkipped": [],
    "reused": false,
    "reportDate": "2026-11-04T09:42:11Z",
    "score": { "value": 612, "band": "Fair", "source": "FIRST_CENTRAL" },
    "summary": {
      "totalLoans": 5, "institutions": 3, "activeLoans": 2, "closedLoans": 3,
      "performingLoans": 1, "delinquentLoans": 1,
      "highestLoanAmount": 2500000.00, "totalBorrowed": 6100000.00,
      "totalOutstanding": 1840000.00, "totalOverdue": 215000.00,
      "maxDaysOverdue": 94, "currency": "NGN"
    },
    "loans": [
      {
        "lender": "FairMoney MFB",
        "accountNumberLast4": "2210",
        "type": "Personal Loan",
```

```
        "amount": 300000.00,
        "outstanding": 215000.00,
        "overdue": 215000.00,
        "status": "OVERDUE",
        "performance": "Delinquent",
        "disbursedDate": "2026-05-02",
        "maturityDate": "2026-08-02",
        "lastPaymentDate": "2026-06-30",
        "tenor": "3 months",
        "repaymentFrequency": "Monthly"
      }
    ],
    "recentEnquiries": [
      { "institution": "Carbon MFB", "reason": "Loan application", "date": "2026-10-19" }
    ]
  },

  "fraudCheck": {
    "status": "OK",
    "riskScore": 38,
    "riskBand": "MEDIUM",
    "recommendation": "Proceed with caution",
    "amlChecked": false,
    "financialAnalysis": {
      "assessed": false,
      "reason": "No loan amount, tenor or income supplied; defaults were used, so these sub-scores are not
meaningful.",
      "items": null
    }
  },

  "bankAccounts": {
    "status": "OK",
    "bvnMatch": "MATCH",
    "clientId": "1001",
    "clientStatus": "Active",
    "kycLevel": "2",
    "customerSince": "2019-02-11",
    "accounts": [
      {
        "accountNumberLast4": "5678",
        "productName": "Farmers Savings",
        "accountType": "SAVINGS",
        "status": "ACTIVE",
```

```
        "rawStatus": "Active",
        "openedDate": "2019-02-11",
        "balance": 142300.55,
        "currency": "NGN",
        "lastTransaction": "2026-11-01",
        "activity": {
          "windowMonths": 12,
          "firstTransaction": "2025-11-04",
          "lastTransaction": "2026-11-01",
          "transactionsInWindow": 87,
          "creditsInWindow": 1850000.00,
          "debitsInWindow": 1702400.00,
          "monthsSinceOpened": 92
        }
      }
    ]
  },

  "bankLoans": {
    "status": "OK",
    "loans": [
      {
        "loanAccountLast4": "9981",
        "productName": "RH-SHF Dry Season Inputs",
        "principal": 2500000.00,
        "outstanding": 1625000.00,
        "overdueAmount": 0.00,
        "daysPastDue": 0,
        "status": "RUNNING",
        "rawStatus": "Active",
        "disbursedDate": "2025-06-10",
        "maturityDate": "2027-06-10",
        "nextDueDate": "2026-11-30",
        "nextDueAmount": 112500.00
      }
    ]
  },

  "crmsFacilities": {
    "status": "OK",
    "matchStatus": "MATCH",
    "summary": { "direct": 2, "indirect": 1, "activeOutstanding": 1625000.00, "balancesComplete": true },
    "relationships": [
      {
```

```
        "role": "BORROWER",
        "track": "RHSHF",
        "caseReference": "RHSHF-2025-00318",
        "facilityType": "Dry-season inputs",
        "obligor": "Grandbai Limited",
        "status": "ACTIVE",
        "amountApproved": 2500000.00,
        "guaranteeAmount": null,
        "outstanding": 1625000.00,
        "daysPastDue": 0,
        "balanceSource": "CORE_BANKING_LIVE"
      }
    ]
  },

  "aiAdvisory": {
    "status": "OK",
    "overallScore": 61.4,
    "overallRating": "Low",
    "recommendation": "ApproveWithConditions",
    "categories": [
      {
        "category": "CreditHistory", "assessed": true, "score": 55.0, "weight": 0.40, "rating": "Medium",
        "rationale": "FIRST_CENTRAL report: 5 loans, 2 active, 1 delinquent, 0 written off, NGN 1,840,000
outstanding. With the bank: 1 running, 0 past due, 2 repaid, 0 written off.",
        "positives": ["2 loans with the bank repaid in full.", "1 running loan with the bank, none past
due."],
        "concerns": ["1 bureau loan is delinquent (NGN 215,000 overdue).", "Up to 94 days overdue at a
bureau."]
      },
      {
        "category": "RepaymentCapacity", "assessed": true, "score": 75.0, "weight": 0.25, "rating": "Low",
        "rationale": "Estimated instalment NGN 48,333 a month on NGN 500,000 over 12 months, plus NGN 112,500
already due monthly with the bank, against average monthly inflow of NGN 420,000 (38%).",
        "positives": ["Obligations would take 38% of monthly inflow."],
        "concerns": ["Existing instalments with the bank of NGN 112,500 a month are counted."]
      },
      {
        "category": "CashflowStability", "assessed": true, "score": 70.0, "weight": 0.20, "rating": "Low",
        "rationale": "87 transactions over 12 months on 1 account. Average inflow NGN 420,000, outflow NGN
390,000 a month.",
        "positives": ["Positive net monthly cashflow of NGN 30,000."],
        "concerns": []
      },
```

```
      {
        "category": "IdentityAndFraud", "assessed": true, "score": 65.0, "weight": 0.15, "rating": "Low",
        "rationale": "BVN verified; watch-list clear; fraud risk 38 (MEDIUM); bank customer yes.",
        "positives": ["BVN verified with the identity provider.", "A customer of the bank for 92 months."],
        "concerns": ["Fraud risk score 38 (medium)."]
      }
    ],
    "redFlags": ["1 delinquent bureau facility."],
    "conditions": ["Clear all overdue facilities before disbursement."],
    "narrative": {
      "summary": "...",
      "strengths": "...",
      "weaknesses": "...",
      "mitigatingFactors": "...",
      "keyRisks": "..."
    },
    "proposal": { "amount": 500000.00, "tenorMonths": 12, "supplied": true, "recommendedAmount": 375000.00,
"recommendedTenorMonths": 12 },
    "basis": "Scored from FIRST_CENTRAL report, statements on 1 account over 12 months, loans with the bank,
identity check, fraud check. The overall score is weighted over the assessed categories only.",
    "narrativeSource": "LLM",
    "modelVersion": "hybrid-v1.0-rules+llm",
    "generatedAt": "2026-11-04T09:42:40Z"
  },

  "coverage": {
    "bankAccounts": "Every account under the customer records core banking holds for this BVN.",
    "bankLoans": "Loans booked in core banking for this customer. Days past due is computed from the repayment
schedule.",
    "creditBureau": "FIRST_CENTRAL answered; no other bureau was called.",
    "fraudCheck": "Overall score only. The financial analysis needs a loan amount, tenor and income.",
    "crmsFacilities": "Every CRMS case the BVN appears on, in any role."
  },
  "warnings": [
    "One bureau loan is 94 days overdue."
  ]
}
```

Appendix B. Values you will meet

Where	Values
Section names	`identity`, `creditBureau`, `fraudCheck`, `bankAccounts`, `bankLoans`, `crmsFacilities`, `aiAdvisory`
Job status	`QUEUED`, `PROCESSING`, `COMPLETED`, `FAILED`
Callback status	`NOT_REQUESTED`, `PENDING`, `DELIVERED`, `FAILED`
Event types	`CUSTOMER_CREDIT_CHECK_COMPLETED`, `CUSTOMER_CREDIT_CHECK_FAILED`
Section status	`OK`, `NO_RECORD`, `UNAVAILABLE`, `NOT_REQUESTED`, `NOT_ALLOWED`
Loan status	`RUNNING`, `OVERDUE`, `PAID`, `WRITTEN_OFF`, `IN_PROCESS`, `UNKNOWN`
Account status	`ACTIVE`, `DORMANT`, `CLOSED`, `INACTIVE`, `UNKNOWN`
Bureaus	`FIRST_CENTRAL`, `CRC`, `CREDIT_REGISTRY`
Fraud risk band	`LOW`, `MEDIUM`, `HIGH`
Advisory rating	`VeryLow`, `Low`, `Medium`, `High`, `VeryHigh`
Advisory recommendation	`StrongApprove`, `Approve`, `ApproveWithConditions`, `Refer`, `Decline`
Role on a CRMS case	`BORROWER`, `GUARANTOR`, `DIRECTOR_OR_OWNER`, `ACCOUNT_SIGNATORY`, `NEXT_OF_KIN_OR_REFEREE`, `OTHER`

New values may be added over time. Write your code so that a value it does not recognise is handled gracefully and does not cause a failure.